

Beispieldokument: Liniendiagramm und Flowchart in LaTeX

Jakob – Abteilung Elektrotechnik, HTBLuVA Salzburg

19. Oktober 2025

1 Liniendiagramm aus CSV-Daten mit pgfplots

1.1 Einleitung

In diesem Abschnitt wird gezeigt, wie mit $\text{\LaTeX}$ und dem Paket `pgfplots` ein Liniendiagramm aus einer externen CSV-Datei erstellt wird. Die Codeausschnitte werden farbig dargestellt und anschließend das fertige Diagramm gezeigt.

1.2 Vorbereitung der Daten

Die zu visualisierenden Daten liegen in einer CSV-Datei vor. Ihr Inhalt sieht wie folgt aus:

```
x,y  
0,1  
1,2  
2,4  
3,3  
4,5  
5,7
```

Listing 1: Inhalt der Datei `daten.csv`

Die erste Zeile enthält die Spaltenüberschriften `x` und `y`. Jede weitere Zeile stellt ein Messpaar dar, das als Punkt im Diagramm visualisiert wird.

1.3 Einlesen und Plotten der Daten

Der zentrale Befehl zum Einlesen der CSV-Daten ist `\addplot` in Kombination mit `table`.

```
1 \addplot [  
2   blue,  
3   mark=*,  
4   line width=1pt  
5 ] table [
```

```

6      col sep=comma,
7      x=x,
8      y=y
9 ] {daten.csv};

```

Listing 2: Plotten der Daten aus einer CSV-Datei

Hier wird die Datei `daten.csv` eingelesen, wobei die Spalte `x` auf die X-Achse und die Spalte `y` auf die Y-Achse abgebildet wird.

1.4 Vollständiges Diagramm

Das folgende Listing zeigt die gesamte `axis`-Umgebung:

```

1 \begin{tikzpicture}
2   \begin{axis}[
3     title={Liniendiagramm aus \texttt{daten.csv}},
4     xlabel={X-Werte},
5     ylabel={Y-Werte},
6     grid=major,
7     legend pos=north west,
8     width=12cm,
9     height=8cm,
10    xmin=0, xmax=5,
11    ymin=0, ymax=8,
12    thick
13  ]
14
15  \addplot[
16    blue,
17    mark=*,
18    line width=1pt
19  ] table [
20    col sep=comma,
21    x=x,
22    y=y
23  ] {daten.csv};
24
25  \legend{Messreihe}
26 \end{axis}
27 \end{tikzpicture}

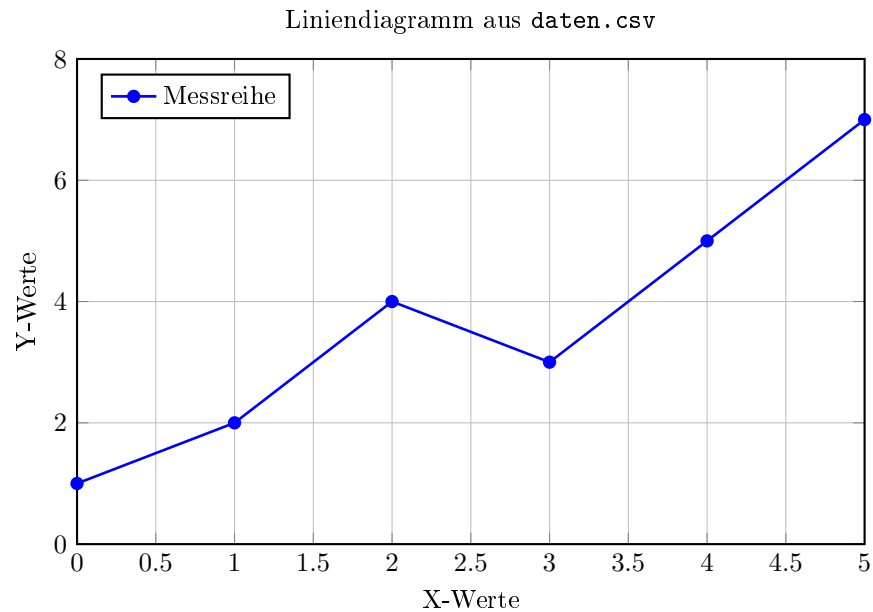
```

Listing 3: Komplette `axis`-Umgebung für das Liniendiagramm

Die Umgebung `axis` definiert das Koordinatensystem. Parameter wie `grid`, `width`, `height` oder `legend pos` bestimmen Darstellung und Layout.

1.5 Ergebnis

Das resultierende Diagramm ist in Abbildung 1.5 dargestellt.



2 Flowchart der Ball Balancing Platform (BBP) mit TikZ

2.1 Einleitung

Im zweiten Teil wird gezeigt, wie mit dem Paket TikZ ein vollständiges Flowchart für die Softwaresteuerung der Ball Balancing Platform (BBP) erstellt wird. Das Diagramm bildet den Programmablauf des Regelkreises ab.

2.2 Definition der TikZ-Styles

Die grafischen Styles bestimmen Form, Farbe und Textausrichtung der Blöcke.

```

1 \tikzstyle{startstop} = [rectangle, rounded corners, minimum
   width=3cm,
2   minimum height=1cm, text centered, text width=3cm,
3   draw=black, fill=red!20]
4 \tikzstyle{process} = [rectangle, minimum width=3cm, minimum
   height=1cm,
5   text centered, text width=3cm, draw=black, fill=orange!20]
6 \tikzstyle{decision} = [diamond, minimum width=3cm, minimum
   height=1cm,
7   text centered, draw=black, fill=green!20, aspect=1]
8 \tikzstyle{io} = [trapezium, trapezium left angle=70,
   trapezium right angle=110,
```

```

9   minimum width=3.5cm, minimum height=1cm,
10  text centered, text width=2cm, draw=black, fill=blue!20]
11 \tikzstyle{arrow} = [thick, ->, >=stealth]

```

Listing 4: Definition der Styles für das Flowchart

2.3 Erstellung der Knoten

Jede Funktionseinheit wird durch einen `node`-Befehl dargestellt. Die Positionierung erfolgt relativ zu anderen Knoten.

```

1 \node(start) [startstop] {Start};
2 \node(homing) [process, below of=start] {Homing};
3 \node(control_loop) [process, below of=homining] {Control Loop
  };
4 \node(camera) [io, below of=control_loop] {Read Camera};
5 \node(circle_detection) [process, below of=camera] {Circle
  Detection};
6 \node(read_mode) [io, below of=circle_detection] {Read Mode
  };
7 \node(mode1) [decision, below of=read_mode, yshift=-0.6cm] {
  Mode = 1?};
8 \node(mode2) [decision, below of=mode1, yshift=-1.2cm] {Mode
  = 2?};
9 \node(target_zero) [process, below of=mode2, yshift=-0.6cm]
  {Set Target = 0};
10 \node(read_joystick) [io, right of=mode1, xshift=6cm] {Read
  Joystick};
11 \node(target_circle) [process, right of=mode2, xshift=2cm] {
  Set Target = Circle()};
12 \node(calc_target) [process, right of=mode2, xshift=6cm] {
  Compute Target Value};
13 \node(pid) [process, below of=target_zero] {PID Control};
14 \node(invkin) [process, below of=pid] {Inverse Kinematics};
15 \node(motor_control) [io, below of=invkin] {Drive Motors};
16 \node(exit) [decision, below of=motor_control, yshift=-0.5cm]
  {Exit?};
17 \node(cleanup) [process, below of=exit, yshift=-0.5cm] {
  Cleanup};
18 \node(stop) [startstop, below of=cleanup] {Stop};

```

Listing 5: Definition der Knoten für das Flowchart

2.4 Verbindungen und Pfeile

Die logischen Verbindungen werden mit `\draw[arrow]` erstellt.

```

1 \draw[arrow] (mode1) -- (mode2) node[anchor=west,
  midway] {No};

```

```

2 \draw[arrow] (mode1) -- (read_joystick) node[anchor=south,
   midway] {Yes};
3 \draw[arrow] (mode2) -- (target_zero) node[anchor=west,
   midway] {No};
4 \draw[arrow] (mode2) -- (target_circle) node[anchor=south,
   midway] {Yes};
5 \draw[arrow] (target_circle) |- (pid); % Knickpfad
6 \draw[arrow] (exit.east) -- node[anchor=south] {No} ++(10,0)
   |- (control_loop.east);

```

Listing 6: Verbindungen der Blöcke mit Beschriftungen

2.5 Ergebnis

Das fertige Flowchart ist in Abbildung 2.5 dargestellt.

